geeksforgeeks.org/problems/prefix-sum-of-matrix-or-2d-array/1?page=1&category=Arrays&sortBy=latest

2 Offers Ending
Search... Courses Tutorials Practice Jobs

Problem | Editorial | Submissions | Comments

Prefix Sum of Matrix (Or 2D Array)

Difficulty: **Medium** Accuracy: **75.71%** Submissions: **140+** Points: **4**
Average Time: **20m**

Given a integer matrix (or 2D array) **a[][]** of dimensions **n * m**. Also, given another 2-D array **query[][]** of dimensions **q * 4**.

For each index 0 < i < query.length, find the sum of all the elements of the rectangular matrix whose top left corner is (query[i][0], query[i][1]) and bottom right corner is (query[i][2], query[i][3]).

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
214 / 214	**1 / 1**

Java (21) Start Timer

```java
// User function Template for Java

class Solution {

    static ArrayList<Long> submatrixSum(long[][] a, int n, int m, int[][] query, int

        long[][] prefix = new long[n][m];

        // Build prefix sum
        for(int i = 0; i < n; i++){
            for(int j = 0; j < m; j++){

                prefix[i][j] = a[i][j];

                if(i > 0)
                    prefix[i][j] += prefix[i-1][j];

                if(j > 0)
                    prefix[i][j] += prefix[i][j-1];

                if(i > 0 && j > 0)
                    prefix[i][j] -= prefix[i-1][j-1];
            }
        }

        ArrayList<Long> ans = new ArrayList<>();

        // Process queries
        for(int i = 0; i < q; i++){

            int n1 = query[i][0];
```

Custom Input | Compile & Run | Submit

geeksforgeeks.org/problems/footpath-construction/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Footpath Construction

Difficulty: **Medium** Accuracy: **40.34%** Submissions: **114+** Points: **4** Average Time: **19m**

Given a matrix **a** of size **n*m** which represents a **park**, there is some construction work needs to be done. You are also given **q** queries each query contains two numbers **R** and **C**, For every query we need to construct a footpath in the **Rth** row and **Cth** column, there is a **cost** of this construction, after the construction this path will divide the park into **sections**, and the cost of the construction is the **sum** of **minimum** value present in all the sections. You are asked to find this cost for all the queries.

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1122 / 1122	**1 / 1**

Java (21) Start Timer

```java
        int[][] bottomLeft = new int[n][m];
        int[][] bottomRight = new int[n][m];

        for(int i = 0; i < n; i++){
            for(int j = 0; j < m; j++){
                int val = a[i][j];
                if(i > 0) val = Math.min(val, topLeft[i-1][j]);
                if(j > 0) val = Math.min(val, topLeft[i][j-1]);
                topLeft[i][j] = val;
            }
        }

        for(int i = 0; i < n; i++){
            for(int j = m-1; j >= 0; j--){
                int val = a[i][j];
                if(i > 0) val = Math.min(val, topRight[i-1][j]);
                if(j < m-1) val = Math.min(val, topRight[i][j+1]);
                topRight[i][j] = val;
            }
        }

        for(int i = n-1; i >= 0; i--){
            for(int j = 0; j < m; j++){
                int val = a[i][j];
                if(i < n-1) val = Math.min(val, bottomLeft[i+1][j]);
                if(j > 0) val = Math.min(val, bottomLeft[i][j-1]);
                bottomLeft[i][j] = val;
            }
        }

        for(int i = n-1; i >= 0; i--){
```

Custom Input Compile & Run Submit

geeksforgeeks.org/problems/minimum-steps-to-halve-sum/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Minimum Steps to Halve Sum

Difficulty: **Medium** Accuracy: **55.59%** Submissions: **14K+** Points: **4**

Given an array **arr[]**, find the **minimum** number of operations required to make the sum of its elements less than or equal to **half** of the original sum. In one operation, you may replace any element with half of its value (with **floating-point** precision).

Examples:

Input: arr[] = [8, 6, 2]
Output: 3

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 1**

Java (21) Start Timer

```java
class Solution {

    int minOperations(int[] arr) {

        PriorityQueue<Double> pq = new PriorityQueue<>(Collections.reverseOr

        double sum = 0;

        for(int x : arr){
            sum += x;
            pq.add((double)x);
        }

        double target = sum / 2.0;
        int steps = 0;

        while(sum > target){
            double val = pq.poll();
            double half = val / 2.0;
            sum -= half;
            pq.add(half);
            steps++;
        }

        return steps;
    }
}
```

Custom Input Compile & Run Submit

Minimum Number of Workers

Difficulty: **Medium** Accuracy: **55.15%** Submissions: **13K+** Points: **4**

You are given an array **arr[]** of size **n** , where arr[i] denotes the range of working hours a person at position **i** can cover.

- If **arr[i]** ≠ **-1**, the person at index i can work and cover the time interval **[i - arr[i], i + arr[i]]**.
- If **arr[i] = -1**, the person is unavailable and cannot cover any time.

The task is to find the **minimum** number of people required to cover the entire

Output Window

Compilation Results | Custom Input

Compilation Completed

• Case 1

Java (21) | Start Timer

```java
class Solution {

    int minMen(int[] arr) {

        int n = arr.length;
        int[] maxReach = new int[n];

        for(int i = 0; i < n; i++){
            if(arr[i] == -1) continue;

            int left = Math.max(0, i - arr[i]);
            int right = Math.min(n - 1, i + arr[i]);

            maxReach[left] = Math.max(maxReach[left], right);
        }

        int workers = 0;
        int currEnd = 0;
        int farthest = 0;
        int i = 0;

        while(currEnd < n){

            while(i <= currEnd){
                farthest = Math.max(farthest, maxReach[i]);
                i++;
            }

            if(farthest < currEnd) return -1;

            workers++;
```

Custom Input | Compile & Run | Submit

Previous Smaller Element

Difficulty: **Medium** Accuracy: **65.83%** Submissions: **8K+** Points: **4**

You are given an integer array **arr[ ]**. For every element in the array, your task is to determine its **Previous Smaller Element (PSE)**.

The Previous Smaller Element (PSE) of an element **x** is the first element that appears to the left of **x** in the array and is strictly smaller than **x**.

Note: If no such element exists, assign **-1** as the PSE for that position.

Examples:

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 1**

Java (21) | Start Timer

```java
class Solution {

    ArrayList<Integer> prevSmaller(int[] arr) {

        int n = arr.length;
        ArrayList<Integer> ans = new ArrayList<>();
        Stack<Integer> st = new Stack<>();

        for(int i = 0; i < n; i++){

            while(!st.isEmpty() && st.peek() >= arr[i]){
                st.pop();
            }

            if(st.isEmpty())
                ans.add(-1);
            else
                ans.add(st.peek());

            st.push(arr[i]);
        }

        return ans;
    }
}
```

Custom Input | Compile & Run | Submit

geeksforgeeks.org/problems/previous-greater-element/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Previous Greater Element

Difficulty: **Medium** Accuracy: **71.97%** Submissions: **6K+** Points: **4**

You are given an integer array **arr[ ]**. For every element in the array, your task is to determine its **Previous Greater Element (PGE)**.

The Previous Greater Element (PGE) of an element **x** is the first element that appears to the left of **x** in the array and is strictly greater than **x**.

Note: If no such element exists, assign **-1** as the PGE for that position.

Examples:

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1**

Java (21) Start Timer

```java
import java.util.*;

class Solution {

    public static ArrayList<Integer> preGreaterEle(int[] arr) {

        int n = arr.length;
        ArrayList<Integer> ans = new ArrayList<>();
        Stack<Integer> st = new Stack<>();

        for(int i = 0; i < n; i++){

            while(!st.isEmpty() && st.peek() <= arr[i]){
                st.pop();
            }

            if(st.isEmpty())
                ans.add(-1);
            else
                ans.add(st.peek());

            st.push(arr[i]);
        }

        return ans;
    }
}
```

Custom Input Compile & Run Submit

Subarrays with First Element Minimum

Difficulty: **Medium** Accuracy: **69.43%** Submissions: **415+** Points: **4**

You are given an integer array **arr[ ]**. Your task is to **count** the number of subarrays where the first element is the **minimum element** of that subarray.

Note: A subarray is valid if its first element is not greater than any other element in that subarray.

Examples:

Output Window

Compilation Results | Custom Input

Compilation Completed

Case 1

Java (21) | Start Timer

```java
import java.util.*;

class Solution {

    public static int countSubarrays(int[] arr) {

        int n = arr.length;
        long count = 0;
        Stack<Integer> st = new Stack<>();

        for (int i = 0; i < n; i++) {

            while (!st.isEmpty() && arr[st.peek()] > arr[i]) {
                st.pop();
            }

            int prevSmallerIndex = st.isEmpty() ? -1 : st.peek();
            count += (i - prevSmallerIndex);

            st.push(i);
        }

        return (int)count;
    }
}
```

Custom Input | Compile & Run | Submit

geeksforgeeks.org/problems/find-132-pattern-in-array/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Find 132 Pattern in Array

Difficulty: **Medium** Accuracy: **71.06%** Submissions: **484+** Points: **4**

You are given an array **arr[].** The task is to determine whether the array contains a **132 pattern**, i.e., three indices i, j and k such that i < j < k , **arr[i] < arr[j] > arr[k]** and **arr[i] < arr[k]**.
Return **true** if such a triplet exists, otherwise return **false**.

Examples:

Input: arr[] = [4, 7, 11, 5, 13, 21

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1**

Java (21) Start Timer

```java
class Solution {
    public boolean has132Pattern(int[] arr) {
        // code here
        int n = arr.length;
        Stack<Integer> st = new Stack<>();
        int third = Integer.MIN_VALUE;

        for(int i = n - 1; i >= 0; i--) {

            if(arr[i] < third)
                return true;

            while(!st.isEmpty() && st.peek() < arr[i]) {
                third = st.pop();
            }

            st.push(arr[i]);
        }

        return false;

    }
}
```

Custom Input Compile & Run Submit

geeksforgeeks.org/problems/maximum-people-visible-in-a-line/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Maximum People Visible in a Line

Difficulty: **Medium** Accuracy: **50.11%** Submissions: **17K+** Points: **4**

You are given an array **arr[ ]**, where arr[i] represents the height of the ith person standing in a line.
A person **i** can see another person **j** if:

- height[j] < height[i],
- There is no person **k** standing between them such that height[k] ≥ height[i].

Each person can see in both directions (front and back).

Output Window

Compilation Results Custom Input

Compilation Completed

Case 1

Java (21) Start Timer

```java
import java.util.*;

class Solution {

    public int maxPeople(int[] arr) {

        int n = arr.length;
        Stack<Integer> st = new Stack<>();
        int third = Integer.MIN_VALUE;

        for(int i = n - 1; i >= 0; i--) {

            if(arr[i] < third)
                return 1;

            while(!st.isEmpty() && st.peek() < arr[i]) {
                third = st.pop();
            }

            st.push(arr[i]);
        }

        return 0;
    }
}
```

Custom Input Compile & Run Submit

Unique K-Number Sum

Difficulty: **Medium** Accuracy: **66.79%** Submissions: **12K+** Points: **4**

Given two integers **n** and **k**, the task is to find all valid combinations of **k** numbers that adds up to **n** based on the following conditions:

- **Only** numbers from the range [1, 9] used.
- Each number can only be used **at most** once.

Note: You can return the combinations in any order, the driver code will print them in sorted order.

Output Window

Compilation Results | Custom Input | Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1120 / 1120	**1 / 1**

Java (21) | Start Timer

```java
class Solution {
    public ArrayList<ArrayList<Integer>> combinationSum(int n, int k) {
        // code here
        ArrayList<ArrayList<Integer>> result = new ArrayList<>();
        backtrack(1, n, k, new ArrayList<>(), result);
        return result;
    }

    private static void backtrack(int start, int target, int k,
                                  ArrayList<Integer> temp,
                                  ArrayList<ArrayList<Integer>> result) {

        if(target == 0 && temp.size() == k) {
            result.add(new ArrayList<>(temp));
            return;
        }

        if(target < 0 || temp.size() >= k)
            return;

        for(int i = start; i <= 9; i++) {
            temp.add(i);
            backtrack(i + 1, target - i, k, temp, result);
            temp.remove(temp.size() - 1);
        }
    }
}
```

Custom Input | Compile & Run | Submit

geeksforgeeks.org/problems/maximize-the-minimum-difference-between-k-elements/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Maximize the minimum difference between k elements

Difficulty: **Medium** Accuracy: **61.58%** Submissions: **13K+** Points: **4**

Given an array **arr[]** of integers and an integer **k**, select k elements from the array such that the **minimum absolute difference** between any two of the selected elements is **maximized**. Return this **maximum** possible **minimum** difference.

Examples:

Input: arr[] = [2, 6, 2, 5], k = 3
Output: 1

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1115 / 1115	**1 / 1**

Java (21) Start Timer

```java
class Solution {
    public int maxMinDiff(int[] arr, int k) {
        // code here
            Arrays.sort(arr);

        int low = 0;
        int high = arr[arr.length - 1] - arr[0];
        int ans = 0;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (canSelect(arr, k, mid)) {
                ans = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return ans;
    }

    private static boolean canSelect(int[] arr, int k, int diff) {

        int count = 1;
        int last = arr[0];

        for (int i = 1; i < arr.length; i++) {

```

Custom Input Compile & Run Submit

geeksforgeeks.org/problems/count-elements-less-than-or-equal-to-k-in-a-sorted-rotated-array/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Count elements less than or equal to k in a sorted rotated array

Difficulty: **Medium** Accuracy: **55.56%** Submissions: **13K+** Points: **4** Average Time: **15m**

Given a sorted array **arr[]** containing distinct non negative integers that has been rotated at some unknown pivot, and a value **x**. Your task is to **count** the number of elements in the array that are **less** than or equal to **x**.

Examples:

Input: arr[] = [4, 5, 8, 1, 3], x = 6
Output: 4

Output Window

Compilation Results Custom Input

Compilation Completed

Case 1

Java (21) Start Timer

```java
class Solution {
    public int countLessEqual(int[] arr, int x) {
        // code here
        int n = arr.length;
        int pivot = findPivot(arr);

        int count = 0;

        count += upperBound(arr, 0, pivot - 1, x);
        count += upperBound(arr, pivot, n - 1, x);

        return count;
    }

    private static int findPivot(int[] arr) {

        int low = 0, high = arr.length - 1;

        while(low < high) {

            int mid = (low + high) / 2;

            if(arr[mid] > arr[high])
                low = mid + 1;
            else
                high = mid;
        }

        return low;
    }

```

Custom Input Compile & Run Submit

geeksforgeeks.org/problems/add-numbers-in-array/1?page=1&category=Arrays&sortBy=latest

Search... 2 Offers Ending Courses Tutorials Practice Jobs

Problem Editorial Submissions Comments

Add numbers in array

Difficulty: **Basic** Accuracy: **86.06%** Submissions: **1K+** Points: **1** Average Time: **2m**

You will be given a vector of integers of size n containing the elements. Your task is to find the sum of all the integers present in the array.As the sum can be large you have to return a value in long long data type.

Example 1:

Input:
n=7

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed	Attempts : Correct / Total
1 / 1	1 / 1

C++ (17) Start Timer

```cpp
//User function Template for C++

class Solution {
public:
    long long get_Sum(int n, vector<int>& input) {

        long long sum = 0;

        for(int i = 0; i < n; i++) {
            sum += input[i];
        }

        return sum;
    }
};
```

Custom Input Compile & Run Submit